东 南 大 学 考 试 卷（A卷）

课程代码	B0402010	课程名称	计算机组织与结构(双语) I	考试学期	24-25-1
适用专业	信息工程	考试形式	闭卷	考试时间	120分钟

题目	I	II	III	IV	V	总分
得分						
批阅人						

学号________ 姓名________ 密 封 线

I. Fill in the blanks（1*18=18'）

1. “Is there a multiple instrument” refers to the ______________ of the computer, and “Is there hardware multiple unit or repeated use of the add unit of the system” refers to the __________ of the computer.

2. ______________ access memory is a ______________ access type of memory to take a comparison of desired bit locations within a word for a specified match, and to do this for all words simultaneously.

3. As a major advantage in computer history, ______________ decouples the architecture of a machine for implementation.

4. For random-access memory, access time (latency) is time from ______________ to the instant that ______________ . And ______________ consists of access time plus additional time required for second access.

5. In a general model of control unit, inputs include ______________ , ______________ , ______________ and ______________.

6. With the development of superscalar machines, strategy has less appeal to deal with branches. Thus, superscalar machines have returned to the techniques of ______________.

7. Unrolling replicates the body of a loop some number of times called ______________ , and iterates by step n instead of step 1.

8. In a microprogrammed control unit, the set of microinstrument is stored in the ____________.

9. In superscalar architecture, decode and execution stages of pipeline are decoupled by a buffer called ______________.

10. A processor has 20 different instruction operations and 16 general-purpose registers. A 32-bit instruction has 1 opcode, 2 register fields and 1 immediate operand field. Number of bits for the operand field is ______________ at most.

II. Multiple Choices（2*5=10'）

1. A 4-byte instruction is stored in memory with start address X. The instruction has an addressing mode of indirect addressing. Address part Y is for operand reference, effective address of the operand is ________ (　　)

   A. X　　B. Y

   C. content in memory of Y　　D. X+Y+4

2. To perform a sequence of 16 instructions by pipeline, each instruction is divided into 4 steps, FI, DI, EI, WO, each stage requires 1 cycle, 1 cycle, 2cycles, 1cycle. There is no branch in the sequence. The speed-up factor of the sequence is ______________ (　　)

   A. 5　　B. 16/7　　C. 4　　D. 64/19

3. Disadvantage of immediate addressing is ______________ (　　)

   A. less memory reference　　B. large value range

   C. limited value range　　D. none of the above

4. Which is **not** the function of computer? (　　)

   A. Data movement　　B. Data creation　　C. Data storage　　D. Control

5. Instrument-level parallelism is not determined by ______________

   A. operation latency　　B. true-data dependency

   C. procedural dependence　　D. resource conflict

III. Translation（15'）

1. Each addressable location in memory has a unique, physically wired-in addressing mechanism. The time to access a given location is independent of the sequence of prior accesses and is constant. Thus, any location can be selected at random and directly addressed and accessed.

2. In the superscalar organization, there are multiple functional units, each of which is implemented as a pipeline. Each individual functional unit provides a degree of parallelism by virtue of its pipelined structure. The use of multiple functional units enables the processor to execute streams of instructions in parallel, one stream for each pipeline.

3. The control unit is the engine that runs the entire computer. It does this based only on knowing the instructions to be executed and the nature of the results of arithmetic and logical operations.

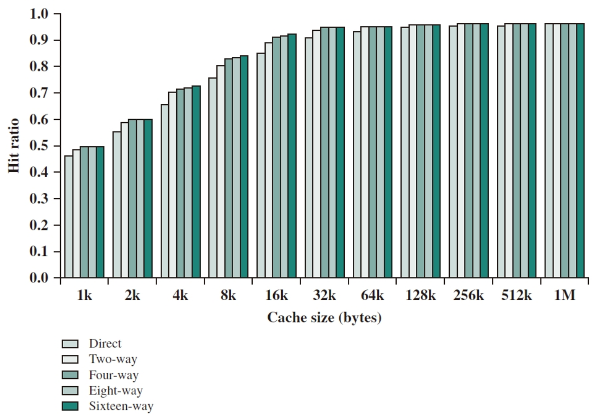
4. To accommodate interrupts, an interrupt cycle is added to the instruction cycle. In the interrupt cycle, the processor checks to see if any interrupts have occurred, indicated by the presence of an interrupt signal. If no interrupts are pending, the processor proceeds to the fetch cycle and fetches the next instruction of the current program.

IV. Review questions（12'）

1. There are 3 address formats for microinstructions. Two address fields, one address field and variable address field. What’s the mean and motivation of variable address field?

2. List key differences between horizontal and vertical microinstruction.

3. Following figure shows the result of simulation study of set-associative cache performance by cache size. What insight can we get from the figure? How should we design cache?



V. Problems（45'）

1. Consider a memory system of 36 bit address of byte level. The cache have 64-byte line and a 256KB capacity.

   （1）Assume the cache is direct mapping cache, show the address format i.e. bits for tag, line and word.

   （2）Assume there is a loop of 20 iterations. A 4-byte word is altered in each loop, making corresponding memory invalid. Write through and write back can fix the problem.

   a. Describe the 2 write policy.

   b. If the write time of a 4-byte word is 30 ns, which policy is more efficient? Give corresponding conditions.

2. Consider a non-pipelined processor with a clock rate of 2.5 GHz. The CPI (cycle per instruction) is 4. Now it’s updated with a 5-stage pipeline, but clock rate drop to 2 GHz, 1 stage per cycle.

   （1）Give the speed up achieved by the upgrade for a typical program of 100 instructions in sequence.

   （2）What’s the MIPS (million instruction per second) rate for each processor (to execute infinite number of instructions)?

3. Consider following code is part of the simulation program, containing 2 loops(L1 and L2). Assume that allocate a taken/not taken switch for each branch instruction. How many prediction errors occur when executing the program, if 1 bit taken/not taken switch is adopted? How many if 2 bit taken/ not taken switch is adopted? Assume that the initial state of the switch is not taken.

Code：

```
S := 0;
for K:=1 to 100 do
for M:=1 to 20 do
S:=S - M;
S:=S - K;
```

A straightforward translation of this into a generic assembly language would look something like this:

```
LD R1, 0 ; keep value of S in R1
LD R2, 1 ; keep value of K in R2
L1
    LD R3, 1 ; keep value of M in R3
    L2
        SUB R1, R1, R3 ; S := S - M
        BEQ R3, 20,L2_EXIT ; done if M = 20
        ADD R3, R3, 1 ; else increment M
        JMP L2 ; back to start of L2
    L2_EXIT
    SUB R1, R1, R2 ; S := S - K
    BEQ R2, 1000, L1_EXIT ; done if K = 1000
    ADD R2, R2, 1 ; else increment K
    JMP L1; back to start of L1
L1_EXIT
```

4. In all instructions in RISC architecture, only LOAD and STORE are memory reference. There are 8 general-purpose registers R1~R8. LOAD/STORE cycle has 3 stages, I, E, D, while other instructions have 2 stages, I, E.

Task: Add a number in memory address A to another number in memory address B, and store the result to memory address C. Then, jump to the instruction at the location of X.

（1）List the instructions to perform the task using RISC, draw a figure to illustrate the pipeline of the instruction sequence.

（2）Optimize the instruction order using techniques you know.

5. The instruction “PUSH rA” is to push the content of the register rA into the stack. The stack is in descending mode, which means the address become smaller when going bottom-up. Give the microoperations and corresponding time unit of fetch and execute cycle of PUSH rA.

6. Consider an in-order issue with in-order completion as is shown below.

Decode		Execution			Write		Cycle
I1	I2						1
	I2			I1			2
	I2			I1			3
I3	I4		I2				4
I5	I6		I4	I3	I1	I2	5
I5	I6			I3			6
		I5	I6		I3	I4	7
		I5					8
					I5	I6	9

（1）Identify why I2 can’t enter execution stage until the forth cycle. Will in-order issue with out-of-order completion and out-of-order issue with out-of-order completion fix it? If so, which?

（2）Identify why I6 can’t enter write stage until the ninth cycle. Will in-order issue with out-of-order completion and out-of-order issue with out-of-order completion fix it? If so, which?

（3）Give better execution and write order by out-of-order issue with out-of-order completion. Give conditions if you need them.

Decode		Execution			Write		Cycle